

modamin / 26S_CST8917

Q

+ ▾

<> CodeIssuesPull requestsAgentsActionsProjectsSecurity and qualityInsights

main ▾

26S_CST8917 / 26S_CT8917_Midterm_Project.md

Q

Go to file

T

⋮

modamin fixed due date

ba61915 · 2 weeks ago

136 lines (90 loc) · 9.04 KB

CST8917 – Serverless Applications

Midterm Project: Smart PDF Analyzer with Durable Functions

Semester	Spring/Summer 2026
Release Date	Friday June 12, 2026 (Week 5)
Due Date	Saturday June 27, 2026 at 11:59 PM (Week 7)
Weight	15% of final grade
Type	Group Assignment (3 or 4 students)

main ▾

26S_CST8917 / 26S_CT8917_Midterm_Project.md

↑ Top

PreviewCodeBlame

Raw

⌵

⋮

Overview

In Lab 2, you built a **Smart Image Analyzer** that used the **Fan-Out/Fan-In** pattern to analyze images in parallel using Azure Durable Functions. In this project, your team will apply the **same architectural pattern** to a new domain: **PDF document analysis**.

You will build a serverless system that automatically processes PDF documents uploaded to Azure Blob Storage. When a PDF is uploaded, your Durable Function will trigger automatically, run four different analyses **in parallel** (Fan-Out/Fan-In), combine them into a unified report (Chaining), and store the results in Azure Table Storage.

IMPORTANT: This is a **coding project**. Your team must implement a fully working Azure Durable Functions application, test it locally with Azurite, and deploy it to Azure.

Requirements

Using the same **hybrid pattern** from Lab 2 (Fan-Out/Fan-In + Chaining), build a PDF analyzer that performs the following **four parallel analyses** when a PDF is uploaded to a `pdfs` Blob Storage container:

#	Analysis Activity	Description
1	<code>extract_text</code>	Extract text content from all pages
2	<code>extract_metadata</code>	Extract PDF metadata (author, title, creation date, etc.)
3	<code>analyze_statistics</code>	Page count, word count, avg words per page, estimated reading time
4	<code>detect_sensitive_data</code>	Scan for emails, phone numbers, URLs, and date patterns

After the fan-in, the results must be combined into a report, stored in **Azure Table Storage**, and retrievable via an **HTTP endpoint**.

Refer to Lab 2 for the overall function structure (client, orchestrator, activities, report, store, HTTP retrieval).

Architecture Diagram

Create a diagram that illustrates the overall architecture of your system, showing how the Blob trigger, orchestrator, parallel activities (Fan-Out/Fan-In), sequential chain (report + store), Table Storage, and HTTP endpoint connect and interact. You may use any diagramming tool (e.g., draw.io, Lucidchart, Excalidraw, Mermaid) and include the diagram in your `README.md`.

Demo Video

Each group must record a **demonstration video (5 to 10 minutes)** covering:

1. **PDF upload (local):** Upload a PDF to Azurite and show the blob trigger firing
2. **Parallel execution:** Show terminal logs with all 4 analysis messages appearing simultaneously
3. **Results retrieval:** Query the HTTP endpoint and show the JSON analysis results
4. **Azure deployment:** Upload a PDF in Azure and retrieve results from the cloud endpoint
5. **Code walkthrough:** Team gives a walkthrough of their project

Requirement	Details
Length	5 to 10 minutes
Participation	All group members must appear and present their part of the code
Upload	YouTube (unlisted is fine)
Link	Include the video link in your <code>README.md</code> file on GitHub

Group Work Guidelines

- Each group consists of **3 students**
- Groups will be created in Brightspace and students will be **randomly assigned**
- Each group is responsible for dividing tasks equally and collaborating to complete the project
- Your submission must reflect the collective effort and understanding of all group members
- **Include a contribution statement** in your `README.md` describing each member's responsibilities

Submission Instructions

Deliverables: Working Azure Functions project (GitHub repo), demo video (YouTube link in `README.md`), and contribution statement.

GitHub Repository must contain:

Item	Description
<code>function_app.py</code>	Complete implementation with all 9 functions
<code>requirements.txt</code>	All dependencies
<code>local.settings.example.json</code>	Template without real connection strings
<code>test-function.http</code>	REST Client file for testing the results endpoint
<code>README.md</code>	Team members, contribution statement, setup instructions, architecture diagram, demo video link, and AI disclosure

GitHub Repository:

1. Create a **private** GitHub repository for your group
2. Add `ramymohamed10` as a collaborator (Settings > Collaborators > Add people)
3. Include all deliverables listed above

Reference: [Inviting collaborators to a personal repository](#)

Submit your **GitHub Repository URL** on Brightspace by the due date.

Warning: It is your responsibility to ensure the instructor is added as a collaborator before the deadline. Submissions without collaborator access at the time of marking will not be accepted.

Security Reminder: Do NOT commit `local.settings.json` with real connection strings. Use `local.settings.example.json` with placeholder values instead.

Marking Rubric

Criteria	Weight	Description
Functionality	30%	All 9 functions work correctly; blob trigger fires on PDF upload; fan-out/fan-in executes in parallel; results stored and retrievable via HTTP endpoint
Code Quality	20%	Clean, well-commented code; proper error handling; meaningful variable names; no hardcoded values
Azure Deployment	20%	Successfully deployed to Azure; PDF upload triggers orchestration in the cloud; results retrievable via cloud endpoint
Demo Video	20%	All 5 required demo items shown; all team members appear and explain their code; within time limit
Documentation	10%	README includes all required sections; contribution statement present; AI disclosure included; setup instructions are clear
	100%	

Academic Integrity

This assignment **permits** the use of generative AI tools (e.g., ChatGPT, GitHub Copilot) to assist with coding, debugging, or research, provided that:

1. **Transparency** — You disclose how AI was used and cite it in your README
2. **Original Thought** — The work primarily reflects your own understanding and effort
3. **Critical Evaluation** — You verify the accuracy and correctness of AI-generated code

Directly submitting AI-generated work without modification, evaluation, or citation violates Algonquin College's Academic Integrity Policy AA48 and could result in a zero or further disciplinary action.

Collaboration **within** your group is expected. Collaboration **between** groups is not permitted. All team members are responsible for understanding the entire codebase — the final oral exam (Weeks 14–15) may include questions about this project.

Late Submission Policy

Late submissions will **not** be accepted. Any changes made to your GitHub repository **after the deadline** will be ignored.